The Publishing API: guide for developers and AI agents

For: a developer building software against it, or an AI agent that schedules posts. Base address: `https://publish.dhakakacchi.com`

An agent can also read the short plain-text version at `GET /llms.txt` and the machine-readable contract at `GET /openapi.json` (interactive page: `/docs`). Neither needs a key. Every rule a post must meet is one document, `GET /v1/spec` (section 5c). AI agents can also connect over **MCP** at `/mcp/` with the same key (section 5f).

1. What it does, in one minute



Posts made here and posts typed in the Google Sheet use the **same engine** (checks, approval, schedule, retries, "never post twice"). They are shown together, sorted by date, in the Sheet's **Calendar** tab (`source` says `agent` for yours). The Sheet never edits or cancels API posts.

2. Get a key

Only the owner can create keys, in their own terminal (the key is shown once, so never send it in chat or email):

```
dk api key-create content-agent --right draft
```

Right	May do
`read`	look at platforms and posts
`draft`	also upload files, validate, create **drafts**, cancel drafts
`approve`	also publish directly (`"approve": true`) and approve drafts

Give an agent the `draft` right. Then every post waits for a human. Use `approve` only for software you fully trust. The owner stops a key at once with `dk api key-revoke content-agent`.

Send the key on **every** `/v1` call:

```
Authorization: Bearer dkp_xxxxxxxxxxxxxxx
```

Set it once in your shell for the examples below (do not paste real keys into scripts that get shared):

```
API=https://publish.dhakakacchi.com
read -rs "KEY?Paste key, press Enter: "; export KEY      # zsh. In bash: read -rs -p "Key: "
KEY; export KEY
```

3. Step by step, with real requests

Step 1: Learn what is possible

```
curl -s -H "Authorization: Bearer $KEY" $API/v1/platforms
```

You get every connected platform with its **formats, caption limits, file types, extra options, and connected accounts**. This is the source of truth and is always current. Do this first, and again whenever a post is refused. One platform: `GET /v1/platforms/instagram`.

Current platforms: `facebook`, `instagram`, `threads`, `youtube`, `tiktok`. `mode` says what happens: `live` = published for real, `assisted` = handed to the owner on Telegram to post by hand (TikTok).

Step 2: Upload each file once

```
curl -s -H "Authorization: Bearer $KEY" -X POST -T ./reel.mp4 "$API/v1/media?filename=reel.mp4"
# -> {"media_id":"3de909cc-...","filename":"reel.mp4","size":8123456,"sha256":"a782..."}
```

Or let the server download a **public https** link:

```
curl -s -H "Authorization: Bearer $KEY" -H "Content-Type: application/json" -X POST
$API/v1/media \
  -d '{"url":"https://example.com/reel.mp4","filename":"reel.mp4"}'
```

Keep the `media_id`. The extension in `filename` matters (it says what the file is). Allowed types: `.jpg .jpeg .png .mp4 .mov .m4v .webm .avi .mkv` (each platform accepts only some, checked in step 3).

Limits: 512 MB per file; each key may keep 5 GB on the server at once (`413 quota_exceeded`); uploads pause with `507 storage_full` when the server disk is nearly full. Files are **deleted a week after their posts are finished**. A post that still names a deleted file is refused with `media_expired`: upload again and use the new id.

Step 3: Check the post (creates nothing)

```
curl -s -H "Authorization: Bearer $KEY" -H "Content-Type: application/json" -X POST
$API/v1/posts/validate -d '{
  "post_key": "eid-2026-reel-1",
  "variants": [
    {"platform": "instagram", "format": "reel", "caption": "Slow-cooked for Eid.",
     "media": ["3de909cc-..."], "slot": "2026-12-20T19:30:00+01:00",
     "options": {"share_to_feed": true}}
  ]
}'
# -> {"valid": true}   or   {"valid": false, "problems": [...], "examples": {...}}
```

Step 4: Create it

Same body, to `POST /v1/posts`. The answer is `201`:

```
{"post_key": "eid-2026-reel-1", "summary": "awaiting_approval", "created_by": "content-agent",
 "variants": [{"variant_id": "...", "platform": "instagram", "account": "Dhaka Kacchi",
"format": "reel",
               "state": "awaiting_approval", "status": "draft", "slot": "2026-12-
20T18:30:00+00:00",
               "handed_over": false}]}
```

The owner is told on Telegram and approves in the Sheet (Calendar tab, tick `approve`).

Step 5: Follow it

```
curl -s -H "Authorization: Bearer $KEY" $API/v1/posts/eid-2026-reel-1
curl -s -H "Authorization: Bearer $KEY" $API/v1/posts          # all API posts, newest first
```

Poll gently (once a minute is plenty), or use a webhook (section 5b). Per variant, `state` is one of:

state	Meaning
`awaiting_approval`	A draft waiting for the owner
`scheduled`	Approved, waiting for the slot
`publishing`	Going out now
`checking`	The outcome is uncertain; the server is verifying with the platform. **Do not resend.**
`published`	Live; `live_url` is set (TikTok `handed_over: true`: the owner posts it by hand)
`failed` / `expired`	It did not go out (`error` explains)
`invalid`	Refused by the platform's rules at approval
`cancelled`	Cancelled

Step 6: Cancel (optional)

```
curl -s -H "Authorization: Bearer $KEY" -X DELETE $API/v1/posts/eid-2026-reel-1
```

Cancels everything that has not gone out. A variant already publishing or published is left and reported as `409`. To **change** a post: cancel it and send a new post with a **new** `post_key`.

4. The post body, field by field

```
{
  "post_key": "eid-2026-reel-1",
  "label": "Eid reel",
  "approve": false,
  "variants": [
    {
      "platform": "instagram",
      "format": "reel",
      "caption": "Slow-cooked for Eid.",
      "media": ["<media_id>"],
      "slot": "2026-12-20T19:30:00+01:00",
      "account": "Dhaka Kacchi",
      "options": {"share_to_feed": true}
    }
  ]
}
```

Field	Rules
`post_key`	**Your own unique id**, 1 to 80 characters: letters, digits and `. _ : -`, starting with a letter or digit. See "Repeating safely".
`label`	Optional internal name. Never published.
`approve`	`false` (default) creates drafts. `true` needs a key with the `approve` right.
`variants`	1 to 20 entries. **One variant = one post on one platform account.** Send several to post the same idea to several platforms.
`platform`, `format`	From `GET /v1/platforms`.
`caption`	The text. Limits are per platform. Some formats have none (YouTube, Instagram story).
`media`	The `media_id`s from step 2, in display order.
`slot`	ISO 8601 **with** an offset: `2026-12-20T19:30:00+01:00` or `...Z`. A time without an offset is **refused** (ambiguous). At least a minute ahead, at most a year.
`account`	Needed only if the platform has more than one connected account.
`options`	The format's extra fields (see below). **Anything platform-specific goes here, not at the top.**

Unknown fields are **refused** (so a typo is caught, not silently ignored). Only one variant per platform and account in a request.

5. Ready-to-send examples for every platform

Each is one variant. Put it inside `"variants": [ ... ]`. Replace `MEDIA_ID` with a real id and pick a future `slot`.

Instagram

```
{"platform":"instagram","format":"feed","caption":"Fresh out of the pot.","media":
["MEDIA_ID"],"slot":"2026-12-20T19:30:00+01:00"}
{"platform":"instagram","format":"carousel","caption":"Three plates.","media":
["MEDIA_ID","MEDIA_ID"],"slot":"2026-12-20T19:30:00+01:00"}
{"platform":"instagram","format":"reel","caption":"Slow-cooked.","media":
["MEDIA_ID"],"slot":"2026-12-20T19:30:00+01:00","options":{"share_to_feed":true}}
{"platform":"instagram","format":"story","media":["MEDIA_ID"],"slot":"2026-12-
20T19:30:00+01:00"}
```

Facebook

```
{"platform":"facebook","format":"post","caption":"Open today from noon.","slot":"2026-12-
20T19:30:00+01:00"}
{"platform":"facebook","format":"photo","caption":"Today's platter.","media":
["MEDIA_ID"],"slot":"2026-12-20T19:30:00+01:00"}
{"platform":"facebook","format":"video","caption":"Watch the dum opening.","media":
["MEDIA_ID"],"slot":"2026-12-20T19:30:00+01:00"}
```

Threads

```
{"platform":"threads","format":"text","caption":"Biryani o'clock.","slot":"2026-12-
20T19:30:00+01:00"}
{"platform":"threads","format":"image","caption":"Look at this.","media":
["MEDIA_ID"],"slot":"2026-12-20T19:30:00+01:00"}
{"platform":"threads","format":"video","caption":"Watch.","media":["MEDIA_ID"],"slot":"2026-12-
20T19:30:00+01:00"}
```

YouTube (no `caption`: use the options)

```
{"platform":"youtube","format":"video","media":["MEDIA_ID"],"slot":"2026-12-20T19:30:00+01:00",
 "options":{"title":"Dum biryani in 40 seconds","description":"How we open the
pot.","made_for_kids":"no"}}
```

TikTok (handed to the owner on Telegram at the slot; `privacy_level` is always required)

```
{"platform":"tiktok","format":"video","caption":"Biryani.","media":["MEDIA_ID"],"slot":"2026-
12-20T19:30:00+01:00",
 "options":{"privacy_level":"public"}}
```

One idea, three platforms, one request

```
{
  "post_key": "eid-2026-1",
  "variants": [
    {"platform":"instagram","format":"reel","caption":"Slow-cooked for Eid.","media":
["MEDIA_ID"],"slot":"2026-12-20T19:30:00+01:00","options":{"share_to_feed":true}},
    {"platform":"youtube","format":"video","media":["MEDIA_ID"],"slot":"2026-12-
20T20:00:00+01:00","options":{"title":"Dum biryani in 40 seconds","made_for_kids":"no"}},
    {"platform":"tiktok","format":"video","caption":"Slow-cooked for Eid.","media":
["MEDIA_ID"],"slot":"2026-12-20T19:30:00+01:00","options":{"privacy_level":"public"}}
  ]
}
```

Cheat sheet (the live, authoritative numbers are in `GET /v1/platforms`)

Platform / format	Files	Caption	Options (* = required)
instagram `feed`	1 JPEG	optional, 2,200	none
instagram `carousel`	2 to 10 JPEG	optional, 2,200	none
instagram `reel`	1 `.mp4` / `.mov`	optional, 2,200	`share_to_feed`, `cover_at_s`
instagram `story`	1 photo or video	none	none
facebook `post`	none	**required**, 63,206	`link`, `delivery` (`direct` / `native`)
facebook `photo`	1 to 10, `.jpg/.png`, 10 MB each	optional	none
facebook `video`	1, up to 1 GB	optional	`delivery`
threads `text`	none	**required**, 500	`reply_control`
threads `image` / `video`	1	optional, 500	`reply_control`
youtube `video`	1	none	`title`, *`made_for_kids`* (yes / no), `description`, `tags`, `category`, `visibility_after_publish`, `delivery`
tiktok `video`	1 `.mp4/.mov/.webm`	optional, 2,200	`privacy_level`* (`public` / `friends` / `followers` / `only_me`), `allow_comments`, `allow_duet`, `allow_stitch`, `commercial_disclosure`

`delivery: "native"` makes Facebook or YouTube hold the post and publish it themselves (the slot must be at least 10 minutes ahead). Default `direct` = this server publishes at the slot.

5b. Webhooks: let the server call you (instead of polling)

Instead of asking "is it published yet?" every minute, give us an address and **we send you a message when a post changes state**. It is optional; polling still works.

1. Register an address (a `draft` key; the address must be `https` and a public host):

```
curl -s -H "Authorization: Bearer $KEY" -H "Content-Type: application/json" -X POST
$API/v1/webhooks \
  -d '{"url": "https://agent.example.com/dk-hooks", "events": ["post.published",
"post.failed"]}'
# -> {"id": "...", "url": "...", "events": [...], "active": true, "secret": "whsec_...",
"note": "Save the secret now..."}
```

Save the `secret` now: it signs every message and is never shown again (leave `events` out to get all of them). A key may have 3 webhooks. List yours with `GET /v1/webhooks`, remove one with `DELETE /v1/webhooks/{endpoint_id}`.

2. Test it (sends one signed message right now and tells you what your server answered):

```
curl -s -H "Authorization: Bearer $KEY" -X POST $API/v1/webhooks/ENDPOINT_ID/ping
# -> {"delivered": true, "http_status": 200}
```

3. What you receive: `POST` to your address with a JSON body

```
{
  "id": "evt_5f0c..._4",
  "type": "post.published",
  "created_at": "2026-12-20T18:30:12+00:00",
  "data": {
    "post_key": "eid-2026-reel-1", "variant_id": "5f0c...", "platform": "instagram",
    "account": "Dhaka Kacchi", "format": "reel", "state": "published", "status": "published",
    "slot": "2026-12-20T18:30:00+00:00", "live_url": "https://www.instagram.com/p/...",
    "handed_over": false, "reason": "published"
  }
}
```

Event	When
`post.approved`	The owner approved it (it is scheduled)
`post.published`	It is live. `data.live_url` is set (TikTok: `handed_over: true`, the owner posts it by hand)
`post.failed`	It did not go out. `data.reason` says why
`post.expired`	It missed its slot and was too late to publish safely
`post.needs_check`	The outcome is uncertain and the server is verifying it. **Do not resend the post**
`post.cancelled`	Cancelled by you, or **rejected by the owner** in the Sheet (`data.reason` says which)
`post.invalid`	The platform's rules refused it at approval
`ping`	A test message from the ping call

Headers: `X-DK-Event` (the type), `X-DK-Event-Id` (the same as `id`, **identical on every retry**), `X-DK-Delivery`, `X-DK-Signature`.

4. Verify the signature, always. Anyone can post to your address; only we know the secret. `X-DK-Signature` looks like `t=1766255412,v1=<hex>`, where `v1` is the HMAC-SHA256, keyed with your secret, of the text `"<t>.<raw request body>"`. Use the **raw bytes** you received (not re-encoded JSON), compare in constant time, and refuse a timestamp older than 5 minutes (a replayed message).

```
import hashlib, hmac, time


def valid(secret: str, header: str, raw_body: bytes) -> bool:
    parts = dict(p.split("=", 1) for p in header.split(","))
    t = int(parts["t"])
    if abs(time.time() - t) > 300:
        return False
    expected = hmac.new(secret.encode(), f"{t}.".encode() + raw_body,
hashlib.sha256).hexdigest()
    return hmac.compare_digest(expected, parts["v1"])
```

```
import { createHmac, timingSafeEqual } from "node:crypto";

export function valid(secret, header, rawBody /* Buffer */) {
  const parts = Object.fromEntries(header.split(",").map((p) => p.split("=")));
  if (Math.abs(Date.now() / 1000 - Number(parts.t)) > 300) return false;
  const expected = createHmac("sha256",
secret).update(`${parts.t}.`).update(rawBody).digest("hex");
  return expected.length === parts.v1.length && timingSafeEqual(Buffer.from(expected),
Buffer.from(parts.v1));
}
```

5. Answer fast, and expect repeats.

- Reply with any `2xx` quickly (do the real work afterwards). Anything else, a timeout or no connection counts as a failure.
- Failed messages are retried after **1 min, 5 min, 30 min, 2 h, 6 h, 24 h**; after the 7th failure we give up and the owner is told on Telegram. The post itself is never affected.
- Delivery is **at least once**: you may get the same message twice. Remember `id` and ignore a repeat.
- Order is not guaranteed (a retry can arrive after a newer event). If unsure, read `GET /v1/posts/{post_key}`.
- Answer `410 Gone` and we switch that webhook off. Redirects are not followed.
- See what happened to each message with `GET /v1/webhooks/{endpoint_id}/deliveries` (`status` is `pending`, `delivered` or `dead`, with `attempts`, `http_status` and `error`).
- Only messages after you registered are sent (no history). Revoking the key stops them.

The owner can list every registered webhook with `dk api webhooks` and switch one off with `dk api webhook-disable ID`.

5c. Build to the contract: `GET /v1/spec`

One document holds every rule a post must meet, built from the same code that checks posts, so it is never out of date. Read it **before making content**: a media maker learns the file types, how many files and the largest size; a writer learns the caption and option limits; code learns the timing and how each platform is published.

```
curl -s "$API/v1/spec?platform=instagram&format=reel" -H "Authorization: Bearer $KEY"
# -> {"spec_version": "3f1c...", "post": {...rules for every post...}, "upload": {...},
"states": {...},
#     "platforms": {"instagram": {"mode": "live", "route": {...}, "accounts": [...], "timing":
{...},
#       "quotas": [...], "formats": {"reel": {"media": {"min": 1, "max": 1, "types": [".mp4",
".mov"]},
#         "caption": {...}, "options": {...}, "example": {...}, "json_schema": {...}}}}}}
```

- Leave out `platform` and `format` for everything. A `read` key is enough.
- `spec_version` changes when any rule, account or setting does. It is also the `ETag`: send `If-None-Match: "<spec_version>"` and you get `304` when nothing changed, so a client can cache it.
- `media.max_bytes` is checked when you submit: a bigger file is refused at once with `file_too_large`, not at the slot.
- `json_schema` lets code check a variant offline. The API's own check (`dry_run` or `/v1/posts/validate`) stays the authority: a few rules, such as a link counting as 23 characters, only the platform's check knows.
- `states` says what every state means and what a client should do in it.
- `dk api spec [--platform ... --format ...]` prints the same document from a terminal.

5d. Many posts at once: `POST /v1/posts/batch`

```
curl -s "$API/v1/posts/batch" -H "Authorization: Bearer $KEY" -H "Content-Type:
application/json" \
  -d '{"dry_run": true, "posts": [ {"post_key": "wk42-reel", "variants": [...]}, {"post_key":
"wk42-text", "variants": [...]} ]}'
# -> {"dry_run": true, "spec_version": "...", "ok": 1, "failed": 1,
#     "results": [{"post_key": "wk42-reel", "ok": true}, {"post_key": "wk42-text", "ok": false,
"problems": [...], "examples": {...}}]}
```

- Up to 20 posts, each exactly like `POST /v1/posts` and each **on its own**: one bad post never stops the others.
- `dry_run: true` checks everything and creates nothing. Then send the same batch without it.
- Sending the same batch again is safe: existing posts come back with `"replayed": true`.
- A batch **always creates drafts**, whatever the key may do. A person approves them in the Calendar.
- A `media` entry may be a `media_id` or a public `https://` link; the server downloads each link once per batch.

5e. Where publishing stands: `GET /v1/overview` and `GET /v1/calendar`

Both cover **every** post, the Google Sheet's too. A `read` key is enough; nothing can be changed through them.

```
curl -s "$API/v1/overview?days_back=7&days_ahead=14" -H "Authorization: Bearer $KEY"
# -> {"pipeline": {"total": 28, "by_state": {"awaiting_approval": {...}, "scheduled":
{"instagram": 9, ...}}},
#     "published": {"total": 44, "by_platform": {...}}, "problems": [...], "next": [...5
slots...],
#     "awaiting_approval": [...], "plan_ends": "2026-10-15T14:30:00+00:00", "note": "Nothing is
scheduled after ..."}
curl -s "$API/v1/calendar?state=published&platform=instagram&limit=20" -H "Authorization:
Bearer $KEY"
# -> {"total": 12, "limit": 20, "posts": [{"post_key", "platform", "account", "format",
"state", "slot", "source",
#                                         "by", "text", "live_url", "error"}, ...]}
```

`/v1/calendar` filters: `state`, `platform`, `source` (`sheet` or `api`), `from` and `to` (ISO 8601 with an offset) and `limit` (at most 200). `state=published` is the history, newest first, with live links. Posts are by slot otherwise. `GET /v1/posts/{post_key}` also shows a Sheet post (reading only: it cannot be cancelled here).

5f. Connect an AI agent over MCP

The same API, for agents that speak MCP (Claude Code, Claude Desktop, Cursor, LangChain, the OpenAI Agents SDK): `https://publish.dhakakacchi.com/mcp/` with the header `Authorization: Bearer <key>`. Same keys, same rights.

Tool	Right	What it does
`publishing_overview`	read	`GET /v1/overview`
`list_posts`	read	`GET /v1/calendar`
`get_post`	read	`GET /v1/posts/{post_key}`
`publishing_spec`	read	`GET /v1/spec`
`submit_posts`	draft	`POST /v1/posts/batch` (always drafts)
`cancel_post`	draft	`DELETE /v1/posts/{post_key}`

- Resources: `dkp://spec` (the whole contract) and `dkp://guide` (`/llms.txt`). Prompt: `publish_posts`.
- There is **no approve tool**, whatever the key may do over HTTP. A person approves in the Calendar.
- Files never travel through MCP: code uploads them with `POST /v1/media`, or a post names a public https link.
- `dk api mcp-config --client claude-code` (or `claude-desktop`, `cursor`, `langchain`, `openai-agents`) prints the settings to paste. The key appears only as the variable `DK_PUBLISH_KEY`, which you set yourself.
- The owner switches MCP on with `"mcp": {"enabled": true}` in `dk.json`. `"disabled_tools": "submit_posts"` switches a tool off.

6. When something is wrong: errors that teach

Every failure is JSON (`application/problem+json`). A validation failure lists **every** problem at once:

```
{
  "status": 422, "title": "The request has problems",
  "problems": [
    {"path": "variants[0].caption", "code": "too_long", "limit": 2200, "actual": 2410,
     "message": "Caption is 2,410 characters; Instagram allows 2,200."},
    {"path": "variants[1].options.privacy_level", "code": "not_allowed",
     "allowed": ["public", "friends", "followers", "only_me"], "message": "..."}
  ],
  "examples": {"variants[0]": {"platform": "instagram", "format": "reel", "media":
["MEDIA_ID"], "slot": "..."}}
}
```

How to react (an agent can do this automatically): read each problem's `path`, fix that field using `limit`, `allowed` or `expected`, copy the shape from `examples`, send again. Nothing was created by a refused request.

HTTP	Meaning
200 / 201	OK / created
401	No key or a wrong key
403	The key lacks the needed right
404	Unknown post
409	Conflict: `post_key_taken`, or a post that cannot be cancelled/approved now
413	`quota_exceeded`: this key's upload space is full
422	The request has problems (listed)
507	`storage_full`: server disk nearly full, uploads paused; tell the owner

Problem `code`s: `invalid`, `invalid_post_key`, `required`, `too_many`, `too_long`, `not_allowed`, `unknown_platform`, `unknown_format`, `unknown_option`, `unknown_field`, `unknown_account`, `account_required`, `unknown_media`, `media_expired`, `media_rejected` (see `expected`), `invalid_slot`, `slot_in_past`, `slot_too_far`, `duplicate_variant`, `post_key_taken`, `unsupported_type`, `too_large`, `file_too_large` (a file over the format's `max_bytes`), `empty`, `webhooks_off`, `filename_required`, `bad_url`, `fetch_failed`, `storage_full`, `quota_exceeded`.

7. Repeating safely (important for agents)

`post_key` is the idempotency key.

- The connection dropped and you do not know if the post was created? **Send exactly the same request again.** You get the existing post back (`200`, header `Idempotent-Replay: true`). It is never created twice.
- The **same `post_key` with different content** is `409 post_key_taken`. To change a post, cancel it and use a new key.
- **Never retry** a post whose state is `checking`: the server is already verifying it with the platform.

- Use stable keys, for example `<campaign>-<platform>-<date>`, so a restarted agent does not create duplicates.

8. Code examples

Python

```
import os, requests

API = "https://publish.dhakakacchi.com"
H = {"Authorization": f"Bearer {os.environ['DK_KEY']}"}  # never hard-code the key

with open("reel.mp4", "rb") as f:  # 1. upload
    media = requests.post(
        f"{API}/v1/media", params={"filename": "reel.mp4"}, headers=H, data=f
    ).json()["media_id"]

post = {  # 2. describe the post
    "post_key": "eid-2026-reel-1",
    "variants": [
        {
            "platform": "instagram",
            "format": "reel",
            "caption": "Slow-cooked for Eid.",
            "media": [media],
            "slot": "2026-12-20T19:30:00+01:00",
            "options": {"share_to_feed": True},
        }
    ],
}
check = requests.post(f"{API}/v1/posts/validate", headers=H, json=post).json()  # 3. check
if not check["valid"]:
    for p in check["problems"]:
        print(p["path"], p["code"], p.get("message"))  # fix these and retry
else:
    print(
        requests.post(f"{API}/v1/posts", headers=H, json=post).json()["summary"]
    )  # 4. awaiting_approval
```

JavaScript (Node 18+)

```
const API = "https://publish.dhakakacchi.com";
const H = { Authorization: `Bearer ${process.env.DK_KEY}`, "Content-Type": "application/json"
};

const post = {
  post_key: "eid-2026-text-1",
  variants: [{ platform: "threads", format: "text", caption: "Biryani o'clock.", slot: "2026-
12-20T19:30:00+01:00" }],
};
const check = await (await fetch(`${API}/v1/posts/validate`, { method: "POST", headers: H,
body: JSON.stringify(post) })).json();
if (!check.valid) console.log(check.problems);
else console.log(await (await fetch(`${API}/v1/posts`, { method: "POST", headers: H, body:
JSON.stringify(post) })).json());
```

9. Notes for an AI agent builder

1. Give the agent **only** a `draft` key, and tell it: *a human approves in the Google Sheet's Calendar tab; never ask for approval rights*.
2. First action every session: `GET /v1/spec` (or `GET /v1/platforms`). Never hard-code limits, they change; keep `spec_version` and send it back as `If-None-Match` to learn cheaply whether anything did.
3. Always call **validate before create**, and apply `problems[].path` fixes. A model can fix its own mistakes from the error text alone.
4. Put platform-specific fields in `options`. Always send `slot` with an offset. Use a stable, unique `post_key`.
5. Never publish "now": the earliest slot is a minute ahead, and drafts need a human anyway.
6. On any network error: resend the **same** request (it is safe), do not invent a new `post_key`.
7. Report back to the human: `post_key`, each platform's `state`, and `live_url` when published.

10. Good to know

- Times: you send any offset; the Sheet and Telegram messages show Berlin time. Times are stored in UTC.
- Posts made here need **connected accounts** (`GET /v1/platforms` lists them). Only the owner can connect accounts.
- Not built yet: editing a post in place (cancel and resend), per-key rate limits on the HTTP API (MCP has its own).
- Questions about a failed post: ask the owner; they see it in Telegram and the Calendar tab.